

OceanIntel Frontend Architecture

Project overview

OceanIntel is a React single-page application for North Indian Ocean surface observations, subsurface temperature reconstruction, forecasts, validation, alerts, maps, and AI-assisted analysis.

Current frontend stack

- ? React 19 and TypeScript 6
- ? Vite 8
- ? React Router 7
- ? Tailwind CSS 4
- ? Leaflet and React Leaflet for 2D maps
- ? Three.js, React Three Fiber, and Drei for 3D ocean visualisation
- ? Recharts for analytical charts
- ? Lucide React for icons
- ? date-fns for date handling

Current application boundaries

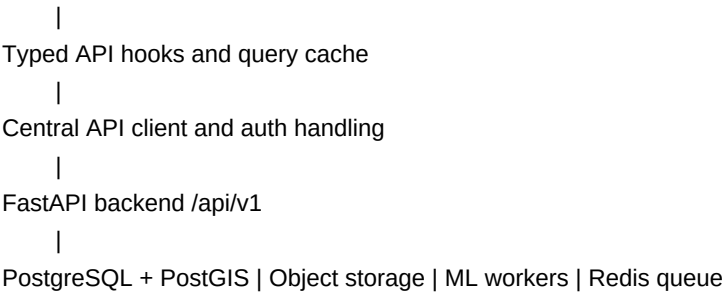
'App.tsx' owns routing, lazy page loading, providers, protected routes, and global effects. 'AuthContext.tsx' owns authentication state. 'DataContext.tsx' owns records, alerts, mock data generation, and localStorage persistence. Pages currently consume these contexts directly.

The current login, NetCDF parsing, reconstruction, forecast, and alert data are mock or client-generated. These are the main boundaries to replace during backend integration.

Target architecture

Pages should render data and trigger actions. They should not perform HTTP requests, parse NetCDF files, run ML inference, or own persistence logic.

React pages and feature components



Recommended folder structure

src/	
app/	App, router, providers, error boundary
config/	Environment variables and constants

api/ API client, errors, query keys, endpoint modules

auth/	Auth provider, user types, permissions, route guards
features/	
surface/	Heatmap, filters, API hooks, domain types
reconstruction/	Profiles, reconstruction jobs, metrics
forecast/	Forecast charts, forecast queries, types
uploads/	NetCDF dropzone, progress, upload jobs
alerts/	Alert list, acknowledgement, audit log
chat/	Chat sessions, messages, streaming responses
validation/	ARGO comparisons and skill metrics
maps/	World and regional map features
components/	
layout/	Navbar and page layout
ui/	Reusable buttons, modals, states, spinners
charts/	Shared chart primitives
maps/	Shared map primitives
ocean/	Domain-specific visual components
hooks/	Generic reusable hooks
types/	Shared API, user, ocean, and pagination types
utils/	Formatting, geography, validation, downloads
styles/	Global styles and design tokens

API client

Create one client in 'src/api/client.ts'. Configure the base URL with 'VITE_API_URL'. The client should handle JSON headers, credentials, token refresh, common errors, request cancellation, and file uploads. Pages and components should call feature API modules rather than 'fetch' directly.

Recommended dependency: TanStack Query. It should own server state, caching, refetching, loading states, retries, mutations, and cache invalidation. Keep React context for authentication and UI state only.

Authentication

Recommended flow:

1. Login with 'POST /api/v1/auth/login'.
2. Keep the access token short-lived and in memory.
3. Store the refresh token in an HttpOnly, Secure, SameSite cookie.
4. Refresh through 'POST /api/v1/auth/refresh'.
5. Load the current user through 'GET /api/v1/auth/me'.
6. Logout through 'POST /api/v1/auth/logout'.

Frontend route guards improve user experience, but every backend endpoint must enforce permissions. Suggested permissions include 'records:read', 'records:write', 'forecasts:read', 'alerts:acknowledge', 'alerts:dispatch', 'validation:read', and 'government:portal'.

Backend API contract

Records and surface data

GET /records
GET /records/latest
GET /records/{recordId}
GET /records/by-date/{date}
POST /records
GET /surface/observations
GET /surface/grid?variable=sst&date=2026-09-05
GET /surface/point?lat=15.5&lon=88&date=2026-09-05

The surface grid response should include variable, unit, date, bounds, resolution, and a two-dimensional values array. The backend should provide the processed grid; the browser should only visualise it. Move the current client-side IDW interpolation out of 'SurfacePage' when real gridded data is available.

Reconstruction and forecasts

POST /reconstructions
GET /reconstructions/{id}
GET /reconstructions/{id}/profile
GET /reconstructions/{id}/metrics
GET /forecasts
GET /forecasts/{id}/profile

Long-running reconstruction and forecast operations should return a job with 'queued', 'processing', 'completed', or 'failed' status and a progress value. The frontend can poll or subscribe to job updates.

Alerts and chat

GET /alerts
POST /alerts
PATCH /alerts/{alertId}/acknowledge
GET /alerts/audit-log
POST /chat/sessions
GET /chat/sessions/{sessionId}/messages
POST /chat/sessions/{sessionId}/messages
POST /chat/sessions/{sessionId}/messages/stream

Use Server-Sent Events or WebSockets for streamed AI responses and live processing status.

Validation

GET /validation/summary
GET /validation/by-depth
GET /validation/argo-collocations
GET /validation/{recordId}

NetCDF upload workflow

1. Frontend calls 'POST /uploads/initiate'.
2. Backend returns a signed object-storage URL.
3. Frontend uploads the NetCDF file directly to object storage.
4. Frontend calls 'POST /uploads/{uploadId}/complete'.
5. Backend queues parsing, validation, and ML inference.
6. Frontend polls 'GET /uploads/{uploadId}/status' or listens for events.
7. Completed records become available through the records API.

Use 'xarray', 'netCDF4', NumPy, Dask, and Zarr on the backend. Avoid sending large NetCDF files through a JSON API.

Core frontend data types

```
interface OceanRecord {
  id: string;
  date: string;
  location: string;
  latitude: number;
  longitude: number;
  inputs: SurfaceInputs;
  profile: DepthProfile;
  mld: number;
  ohc: number;
  thermoclineDepth: number;
  modelVersion: string;
  dataSources: string[];
  processedAt: string;
}
```

Represent missing ARGO values as 'null', not by filtering arrays. Depths, temperatures, and validation metrics must retain identical indexes.

State ownership

```
| State | Owner |
|---|---|
| Authenticated user | Auth provider |
| Records, forecasts, alerts | TanStack Query |
| Upload progress | Upload feature hook |
| Chat session and messages | Chat feature and backend |
| URL filters | React Router search params |
| Form values | Local state or React Hook Form |
| Tabs, modals, selected map point | Local component state |
```

Suggested backend technology

? FastAPI and Pydantic for the API

? PostgreSQL with PostGIS for users, records, locations, and spatial queries

- ? TimescaleDB for high-volume time-series data
- ? S3-compatible storage for NetCDF and model artifacts
- ? PyTorch for inference
- ? Redis with Celery, Dramatiq, or RQ for background jobs
- ? xarray, netCDF4, NumPy, pandas, Dask, and Zarr for scientific data processing

Migration sequence

1. Add 'src/api/client.ts', environment configuration, and shared API types.
2. Replace mock login with the authentication endpoints.
3. Add TanStack Query and create records and alerts API hooks.
4. Replace 'DataContext.records' with query-backed records.
5. Move profile generation and forecasts to backend services.
6. Integrate signed NetCDF uploads and processing jobs.
7. Add alert mutations and audit history.
8. Add chat sessions and streamed responses.
9. Add consistent loading, empty, error, retry, and permission states.
10. Generate TypeScript types from the backend OpenAPI schema.

The first high-value refactor is separating API access from 'AuthContext' and 'DataContext'. This allows the existing interface to migrate page by page without rewriting the visual layer.